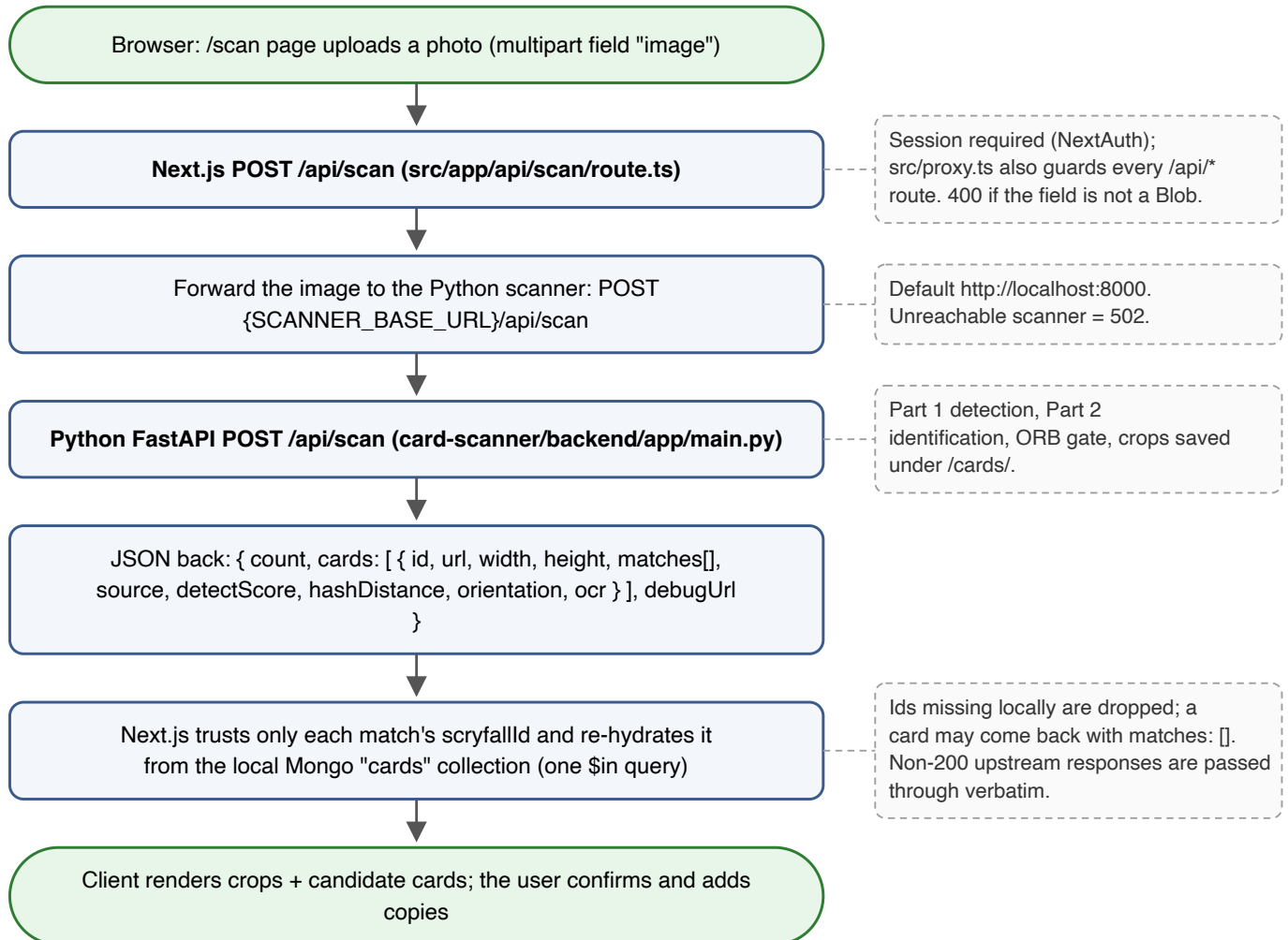


Card scanning: from upload to result

What happens when the app receives a photo of Magic cards. Two services take part: the Next.js app (auth, proxying, and re-hydrating matches from the local card database) and the Python **card-scanner** service (finding the cards in the photo, identifying each one). Everything below is taken from the code as of 14 September 2026.

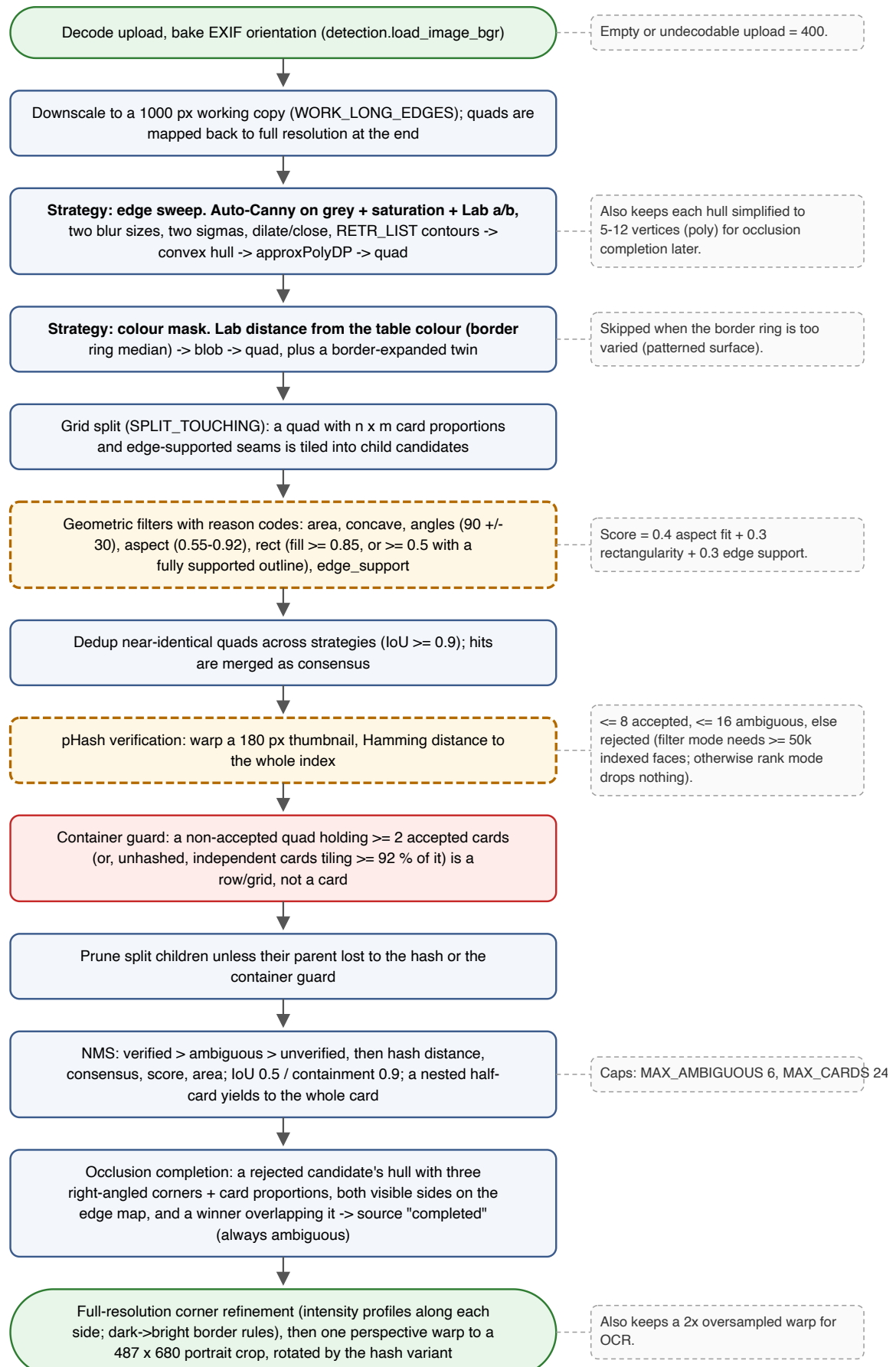
1. The big picture



Files: `src/app/api/scan/route.ts` (proxy + enrichment), `card-scanner/backend/app/main.py` (scanner endpoint), `src/types/ScanResult.ts` (response contract).

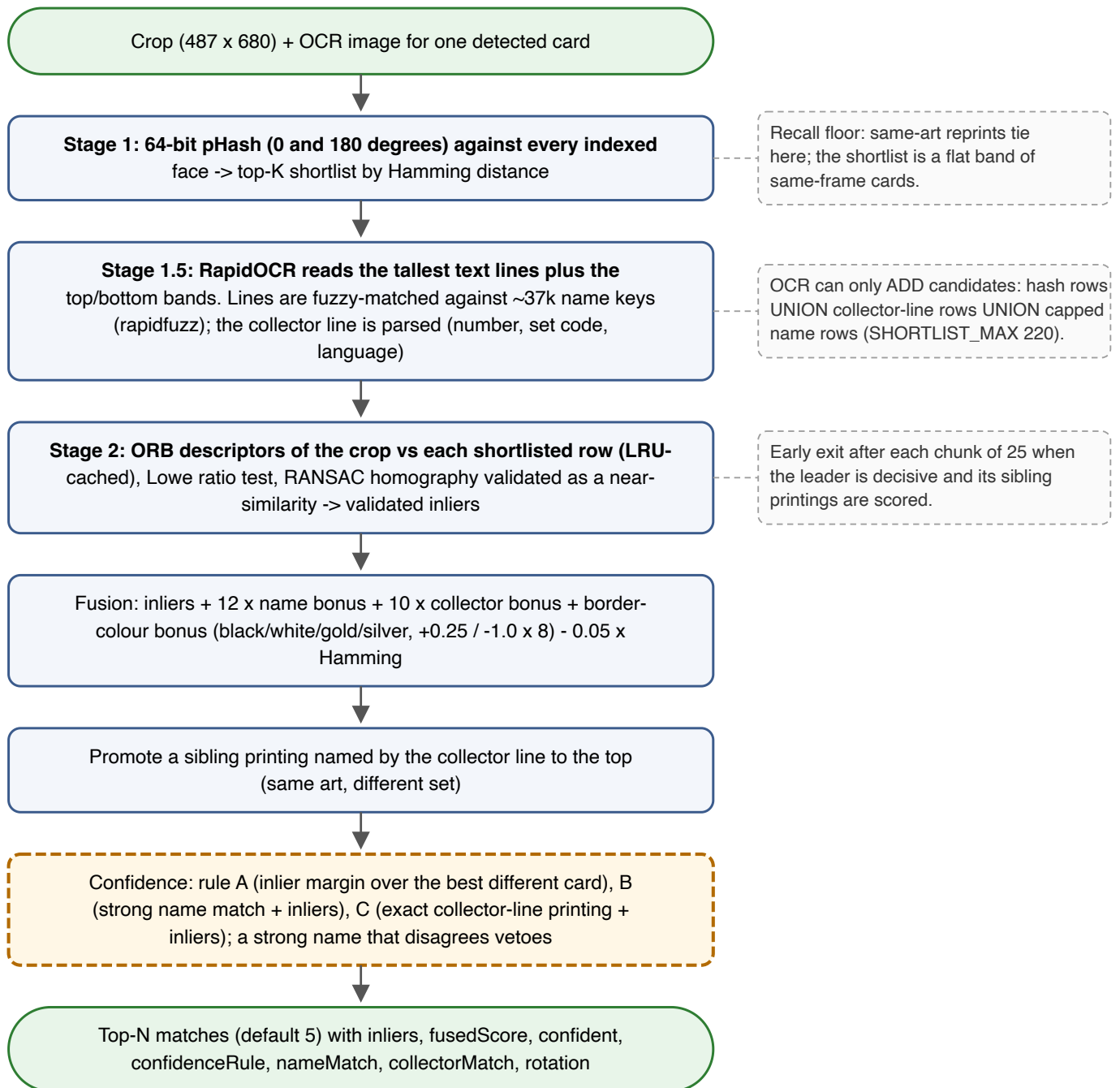
2. Part 1 - Detection: finding the cards in the photo

`detection.detect()` turns one photo into a list of de-skewed card crops. It is a classical pipeline: several cheap strategies propose quadrilaterals, geometric filters and the image index throw out what is not a card, and the survivors are refined at full resolution.

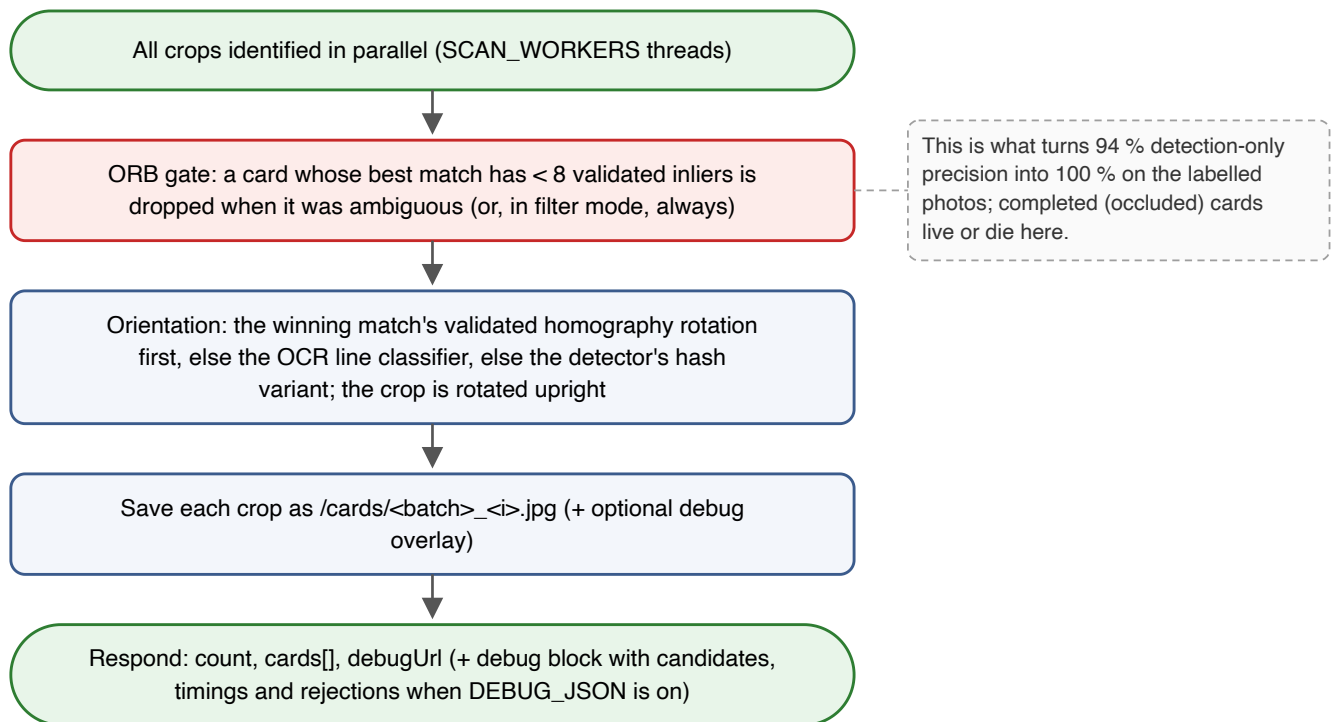


3. Part 2 - Identification: which printing is this?

`matcher.identify_card()` runs on each crop. Three stages trade recall for precision, and text read by OCR is fused with image evidence rather than used as a filter, so an unreadable name never loses a card.



4. The gate, the crops and the response



5. Where the numbers stand (17 labelled photos, 68 cards)

Metric	Value
Detection recall	95.6 % (65/68); 98.5 % of cards not under another card
Precision after the ORB gate	100 %
Identified at card level	65/65 (86 % to the exact printing; the rest are inherent twins)
Detection latency	~330 ms per 12 MP photo
Identification latency	~370 ms per crop, of which OCR ~200 ms
Corner error	median 5 px, p95 6.7 % of card width

Where it still fails

- A borderless card on a busy playmat: no closed outline for any strategy.
- A card with fewer than three visible corners under other cards.
- Corner precision on glare-bright foil borders and on black borders over equally dark wood (outside the dark-border model).
- Printings that are twins by design: The List reprints carrying the original's collector line, prerelease promos, World Championship gold-border reprints.

How it is measured

`app.evaluate_photos` runs the real pipeline on the hand-labelled photos in `card-scanner/test-images/` and diffs against committed baselines in `card-scanner/benchmarks/` (one with the index, one without for CI); `app.label_photos -refine` snaps labels to the card's Scryfall image.

6. Error paths

Where	Condition	Result
Next.js	No session	401
Next.js	No "image" field / not a Blob	400
Next.js	Scanner unreachable	502 "Card scanner unavailable"
Scanner	Empty or undecodable upload	400 (passed through verbatim)
Scanner	Identification throws for one crop	That crop gets matches: [] ; the scan still succeeds
Scanner	Index empty (not built yet)	Cards detected in rank mode, no matches returned
Next.js	Match id not in local cards	Match dropped silently